



Employee Blocks Layout Documentation

1. Introduction

This document outlines the transition from the **Employee Carousel** to a **Block-style Employee Grid** with pagination.

The new design improves readability, navigation, and scalability for larger employee datasets.

Note: Employee data is stored directly in the Angular component, not in a database.

2. Overview of Changes

- **Previous Design:** Auto-scrolling vertical carousel of employees.
- **New Design:** Static block-style grid with pagination controls.
- **Benefits:**
 - Easier navigation and readability.
 - Responsive design for different screen sizes.
 - Pagination for better organization of large datasets.
 - Cleaner UI/UX for professional applications.

3. Features

- **Grid Layout:** Employees displayed in responsive blocks.
- **Pagination:** Configurable number of employees per page.
- **Modal Popup:** Clicking a block opens detailed employee information.
- **Responsive Design:** Works across desktop, tablet, and mobile.
- **Customizable Styling:** Colors, fonts, and branding can be adapted.
- **Data Handling:** Employee data is stored in Angular arrays, not in a backend database.

4. Usage Guidelines

- **Adding Employees:** Extend the employee dataset in the Angular component.
- **Changing Page Size:** Adjust the pagination setting to show more or fewer employees per page.
- **Styling:** Customize the block design to match corporate branding.
- **Modal Content:** Extend employee details with department, email, phone, etc.

5. Future Enhancements

-  **Search Filter:** Allow users to search employees by name/designation.
-  **Mobile Optimization:** Enhance responsiveness for smaller screens.
-  **Sorting Options:** Sort employees by department, role, or seniority.
-  **API Integration:** Optionally fetch employee data dynamically from backend services (if needed in future).

6. Visuals (to be added)

- **Grid Layout Screenshot**
- **Pagination Controls Screenshot**
- **Modal Popup Screenshot**

7. Conclusion

The **Employee Blocks Layout** replaces the carousel with a more structured, user-friendly design.

It provides scalability, clarity, and flexibility for future enhancements, making it a better fit for modern employee management systems.

By storing data in Angular components rather than a database, the solution remains lightweight and easy to maintain for prototype or small-scale applications.

How To Show Last Year Employee Data WITHOUT Database

You must store historical data inside Angular manually.

There are 3 approaches:

◆ OPTION 1: Store Year Field Inside Employee Object

Add a **year** property.

```
employees = [  
  {  
    id: 1,  
    name: 'Ravi Kumar',  
    designation: 'Frontend Developer',  
    department: 'IT',  
    salary: 50000,  
    year: 2025  
  },  
  {  
    id: 2,  
    name: 'Anita Sharma',  
    designation: 'HR Manager',  
    department: 'HR',  
    salary: 70000,  
    year: 2024  
  }  
];
```

Filter by Year

```
selectedYear = 2024;

filteredEmployees: any[] = [];

filterByYear() {

  this.filteredEmployees = this.employees.filter(

    emp => emp.year === this.selectedYear

  );
}
```

HTML

```
<select [(ngModel)]="selectedYear" (change)="filterByYear()">

  <option value="2024">2024</option>

  <option value="2025">2025</option>

</select>


<div *ngFor="let emp of filteredEmployees">

  {{ emp.name }} - {{ emp.designation }}

</div>
```

✓ This will show last year employees.

◆ **OPTION 2: Store Complete History Inside Object (Better Approach)**

If you want full historical details:

```
employees = [  
  {  
    id: 1,  
    name: 'Ravi Kumar',  
    history: [  
      {  
        year: 2024,  
        designation: 'Junior Developer',  
        salary: 40000  
      },  
      {  
        year: 2025,  
        designation: 'Frontend Developer',  
        salary: 50000  
      }  
    ]  
  }  
]
```

```
];
```

Get Data For Specific Year

```
getEmployeeDataByYear(employee: any, year: number) {  
    return employee.history.find(h => h.year === year);  
}
```

Display

```
<div *ngFor="let emp of employees">  
    <div *ngIf="getEmployeeDataByYear(emp, selectedYear) as yearlyData">  
        {{ emp.name }}  
        {{ yearlyData.designation }}  
        {{ yearlyData.salary }}  
    </div>  
</div>
```

🔥 This is much better for CRM-like UI.

◆ **OPTION 3: Use LocalStorage (Still No Database)**

You can store employee data in browser:

```
localStorage.setItem('employees', JSON.stringify(this.employees));
```

Retrieve:

```
this.employees = JSON.parse(localStorage.getItem('employees') || '[]');
```